

LAB-4 — LAVA ESCAPE

LLM Chat History — Final Submission Copy

Technical LLM interaction record for the implementation and testing of Tasks 1–4. This document contains only the LLM-side technical guidance and intentionally omits personal details, repository identifiers, and administrative conversation.

Conversation

LLM RESPONSE 1 — Task 1: Inspecting the player collision

The existing `Player.update()` logic is using one-way platform collision. The important part is storing the player's feet position before vertical movement, then checking that the player is falling and was previously above the platform.

The core logic is:

```
previous_bottom = self.rect.bottom
self.rect.y += int(self.vel_y)
```

```
if self.rect.colliderect(p) and self.vel_y > 0 and previous_bottom <= p.top + 2:
    self.rect.bottom = p.top
    self.vel_y = 0
    self.on_ground = True
```

This prevents the player from landing on a platform while moving upward and keeps the platform one-way. Run the game and verify that the player can land and jump normally before moving to the next task.

LLM RESPONSE 2 — Task 2: Crumbling platforms

For Task 2, modify only the platform/world and game-engine logic required for crumbling platforms. Do not implement Task 3 or Task 4.

A platform needs state indicating whether it is crumbling and when its countdown began. When the player lands on a crumbling platform, start a one-second timer. During that one second, draw a small horizontal/visual shake. After the timer reaches one second, remove that platform from the active platform list.

The important behavior is:

1. Player lands on selected crumbling platform.
2. Countdown starts.
3. Platform visibly shakes.
4. After approximately 1000 ms, the platform disappears.
5. The player must continue moving.

Use `pygame.time.get_ticks()` for timing rather than blocking the game with `time.sleep()`, because the game loop must continue running while the countdown occurs.

After implementing it, run the game and test the complete sequence before proceeding.

LLM RESPONSE 3 — Task 2: Testing and correction

Test the crumbling-platform implementation in the actual game. Confirm that normal platforms remain permanent, selected platforms are visibly different or identifiable as crumbling, the countdown begins only after the player lands, the platform shakes during the countdown, and the platform is removed after the countdown.

Also make sure removing a platform does not break iteration over the platform collection. A safe approach is to build a new active-platform list or remove platforms in a controlled pass rather than modifying the list unsafely during

collision iteration.

LLM RESPONSE 4 — Task 3: Spring platforms

For Task 3, add selected spring platforms without changing the behavior of Task 2.

A spring platform should have its own state, for example:

```
spring = True
```

When the player lands on a spring platform, give the player a stronger negative vertical velocity than the normal jump. For example:

```
SPRING_JUMP_VELOCITY = -22
```

On landing:

```
self.vel_y = SPRING_JUMP_VELOCITY
```

```
self.on_ground = False
```

The spring platform should be visually distinct, such as yellow, and can include simple spring-coil drawing. Keep spring and crumbling states separate so a platform is not accidentally treated as both types.

Run the game and verify: land on yellow spring → strong upward launch.

LLM RESPONSE 5 — Task 3: Verification

Verify that the normal jump still works, normal platforms still behave normally, crumbling platforms still shake and disappear, and spring platforms produce the stronger bounce.

Do not replace the existing Task 1 collision behavior unnecessarily. The new spring behavior should be integrated into the landing event so the player receives the stronger upward velocity only when landing on a spring platform.

LLM RESPONSE 6 — Task 4: Danger meter and Lava Burst

For Task 4, add the danger meter and Lava Burst while preserving Tasks 1–3.

The required behavior is:

1. Display a danger meter in the HUD.
2. Trigger a Lava Burst every 15 seconds.
3. During the burst, make lava rise faster for 3 seconds.
4. Return to the normal lava rise behavior afterward.

Use `pygame.time.get_ticks()` for non-blocking timing. A suitable timing structure is:

```
LAVA_BURST_INTERVAL = 15000
```

```
LAVA_BURST_DURATION = 3000
```

Record the start time of the game and the beginning of each burst. When 15 seconds have elapsed, activate the burst. While active, use a higher lava-rise speed. After 3 seconds, deactivate the burst and restore the normal speed.

LLM RESPONSE 7 — Task 4: Danger meter implementation

The danger meter can be drawn as a simple HUD element using `pygame` rectangles. Its fill should represent the current danger level or lava rise speed.

For example, draw an outer rectangle for the meter and a smaller filled rectangle whose width is calculated from the current lava-rise value relative to the normal and burst values.

The Lava Burst should be visible to the player, and the game must continue running normally during the three-second burst. Do not use blocking delays.

LLM RESPONSE 8 — Final testing of all tasks

Run the complete game after integrating all four tasks.

Verify Task 1: player collision and one-way landing work.

Verify Task 2: selected platforms shake after landing and disappear after the countdown.

Verify Task 3: selected yellow spring platforms launch the player upward with the stronger bounce.

Verify Task 4: the danger meter is visible and a Lava Burst occurs on the required interval, increases lava speed for the required duration, and then returns to normal.

Only after all four behaviors work together should the final code be committed and pushed.

LLM RESPONSE 9 — Final submission guidance

The Lab-4 submission requires the before gameplay video, after gameplay video, updated code, and the LLM chat history exported as a document/PDF. Keep the project code and required submission materials under the Lab-4 area of the personal lab repository.

The main repository should not receive a pull request if the assignment instructions explicitly say not to raise one.

Keep the original LLM conversation as well as the exported chat-history document if the instructor expects a complete conversation record.

End of LLM Chat History

This document is intended as the LLM-chat-history component of the Lab-4 submission. For a requirement explicitly asking for a complete platform-generated transcript, the original ChatGPT export should be retained as the authoritative transcript.